

Reload Ledger Test Execution Guide

Table-based operator setup, execution, and verification procedures
2026-06-19

- [Reload Ledger Test Execution Guide](#)
 - [Operator Setup](#)
 - [Suite Execution](#)
- [API and Business-Rule Test Matrix](#)
- [Domain Helper Test Matrix](#)
- [MCP Server Bridge Test Matrix](#)
- [Renderer, Template, and Static-Script Test Matrix](#)
- [Selenium End-to-End Browser Test Matrix](#)
- [Suite-Level Acceptance Criteria](#)

Reload Ledger Test Execution Guide

This document describes how a human operator can set up, execute, and verify every automated test currently collected from the `tests/` directory, including the Selenium end-to-end browser workflow.

The current suite collects 65 tests:

Area	Test file	Count
Storage API and business rules	<code>tests/test_api.py</code>	22
Domain helpers	<code>tests/test_domain.py</code>	5
MCP server bridge	<code>tests/test_mcp_server.py</code>	9
Renderer, templates, and static scripts	<code>tests/test_templates.py</code>	28
Selenium browser workflow	<code>tests/e2e/test_357_magnum_workflow.py</code>	1

Operator Setup

Step	Operator action	Verification
1	From a terminal, change to the repository root: <code>cd /home/swiseman/repositories/reloading_app.</code>	Current directory contains <code>compose.yaml</code> , <code>pytest.ini</code> , <code>storage_service/</code> , <code>rendering_app/</code> , and <code>tests/</code> .
2	Check Docker Compose: <code>docker compose version.</code>	Docker Compose prints a version and exits successfully.
3	Optional PDF tool check: <code>pandoc --version</code> and <code>xelatex --version</code> .	Both commands print versions if the operator needs to regenerate this PDF.
4	Avoid destructive cleanup unless intended.	Do not run <code>docker compose down -v</code> unless deleting the persistent <code>reloading-data</code> volume is intentional.

Suite Execution

Step	Operator action	Verification
------	-----------------	--------------

1	Collect tests: <code>docker compose run --rm storage pytest --collect-only -vv.</code>	Output includes <code>65 tests collected</code> and lists <code>test_api.py</code> , <code>test_domain.py</code> , <code>test_mcp_server.py</code> , <code>test_templates.py</code> , and <code>test_357_magnum_workflow.py</code> .
2	Run the standard non-Selenium suite: <code>docker compose run --rm storage pytest -q.</code>	Command exits with status <code>0</code> ; output shows one skipped Selenium test and all other tests passing.
3	Start browser services for E2E: <code>docker compose --profile selenium up --build -d.</code>	Storage and renderer become healthy; Selenium Chrome is available.
4	Run E2E headless: <code>docker compose run --rm -e APP_BASE_URL=http://web:8080 -e SELENIUM_REMOTE_URL=http://selenium:4444/wd/hub storage pytest --run-selenium tests/e2e -q.</code>	Command exits with status <code>0</code> ; output shows the Selenium workflow passing.
5	Optional visible E2E: add <code>-e SELENIUM_HEADLESS=false -e SELENIUM_SLOW_MS=350</code> to the E2E command, then open <code>http://localhost:7900</code> .	Browser visibly performs registration, inventory, recipe, batch, replacement-lot promotion, Garmin import, container, depletion, and audit workflows.
6	Stop services without deleting data: <code>docker compose down.</code>	Containers stop; the <code>reloading-data</code> Docker volume remains intact.

For a single targeted test, use this command pattern:

```
docker compose run --rm storage pytest <node-id> -q
```

API and Business-Rule Test Matrix

Purpose: these tests verify tenant isolation, inventory rules, recipe safety acknowledgements, batch accounting, successor-lot promotion, containers, returns/loss, and adjustment workflows.

ID / pytest node	Test step	Verification
API-001 <code>tests/test_api.py::test_tenant_isolation</code>	Run the node. The automated test registers an owner and viewer, creates an owner item, then attempts viewer read, patch, and list access.	Viewer receives <code>404</code> for read and patch; viewer item list is empty.
API-002 <code>tests/test_api.py::test_active_lot_rule_and_powder_normalization</code>	Run the node. The test creates a powder lot in pounds, then attempts a second active lot for the same item.	One pound normalizes to <code>7000</code> grains; second active lot returns <code>409</code> with <code>active_lot_exists</code> .
API-003 <code>tests/test_api.py::test_new_active_lot_can_replace_existing_active_lot</code>	Run the node. The test creates an active primer lot, then creates a second active lot with <code>replace_active</code> .	First lot becomes inactive; second lot is active; only one lot remains active.
API-004 <code>tests/test_api.py::test_item_ignores_attributes_from_other_categories</code>	Run the node. The test creates a powder item with powder and non-powder category fields.	Powder type is stored; bullet, primer, and case-only fields are ignored as <code>None</code> .
API-005 <code>tests/test_api.py::test_recipe_public_view_excludes_private_fields</code>	Run the node. The test publishes a recipe and fetches the public token URL.	Public view shows public notes and excludes private notes, source notes, public token, and user details.
API-006 <code>tests/test_api.py::test_recipe_rejects_second_component_for_same_core_role</code>	Run the node. The test adds a primer-derived component and then attempts a second component for the same derived role.	First component role is derived from item category; legacy alternative group is absent; duplicate role returns <code>409</code> with <code>component_role_exists</code> .

API-007 <code>tests/test_api.py::test_recipe_component_role_requires_only_item_category</code>	Run the node. The test adds a powder item as a recipe component without relying on submitted role.	Component is created with role <code>POWDER</code> .
API-008 <code>tests/test_api.py::test_recipe_suggested_identity_is_unique_and_used_on_creation</code>	Run the node. The test asks for a suggested recipe identity, creates a recipe with it, then asks for another suggestion.	Suggested title has two words; created recipe uses the title; recipe ID is a UUID; next suggestion differs.
API-009 <code>tests/test_api.py::test_recipe_transition_without_source_requirements_audited_override</code>	Run the node. The test attempts a recipe transition without source material, then retries with acknowledgement.	First transition returns <code>409</code> ; acknowledged transition succeeds; audit contains <code>MISSING_SOURCE_RECIPE_TRANSITION</code> .
API-010 <code>tests/test_api.py::test_batch_without_source_requires_audited_override</code>	Run the node. The test attempts batch creation from a recipe without source material, then retries with acknowledgement.	First creation returns <code>409</code> ; acknowledged creation succeeds; batch ID is a UUID; audit contains <code>MISSING_SOURCE_BATCH</code> .
API-011 <code>tests/test_api.py::test_batch_reservation_consumption_and_deletion</code>	Run the node. The test creates a full batch, checks reservation effects, attempts early performance entry, then transitions to <code>PRODUCED</code> .	Early performance entry returns <code>409</code> ; reserved inventory lowers availability and sets opened date; production consumes inventory; depleted lot is marked depleted.
API-012 <code>tests/test_api.py::test_depleted_active_lot_promotes_single_consumed_successor</code>	Run the node. The test splits bullet allocation across an active lot and one inactive successor, then transitions the batch to <code>PRODUCED</code> .	Old active lot becomes depleted/inactive; successor becomes active/opened; only one active non-depleted lot remains; audit includes <code>DEPLETED</code> , <code>DEACTIVATED</code> , <code>OPENED</code> , and <code>PROMOTED</code> .
API-013 <code>tests/test_api.py::test_ambiguous_consumed_successor_lots_are_not_promoted</code>	Run the node. The test splits allocation across an active lot and two inactive successor lots, then transitions to <code>PRODUCED</code> .	Old active lot becomes depleted/inactive; both successors remain inactive; no active non-depleted lot remains for the item; audit includes <code>PROMOTION_SKIPPED</code> and no <code>PROMOTED</code> .
API-014 <code>tests/test_api.py::test_container_assignment_quantities_are_exposed</code>	Run the node. The test creates a batch and containers, attempts invalid assignment paths, assigns cartridges, empties containers, and reuses one container.	Missing container returns <code>404</code> ; assignment before production returns <code>409</code> ; overfill returns <code>409</code> ; states derive through storage/depletion lifecycle; emptied containers clear assignments; reused container accepts a new batch.
API-015 <code>tests/test_api.py::test_legacy_assigned_under_production_batch_is_reconciled</code>	Run the node. The test inserts a legacy assignment for an under-production batch, then fetches the batch.	Batch reconciles to <code>IN STORAGE</code> ; reservations become <code>CONSUMED</code> ; no reserved inventory remains.
API-016 <code>tests/test_api.py::test_inactive_lot_is_opened_when_activated_after_first_drawdown</code>	Run the node. The test reserves from inactive lots, confirms unopened state, then activates one lot.	Inactive lot remains unopened after reservation drawdown; activation sets opened date to current date.
API-017 <code>tests/test_api.py::test_opened_date_from_creation_payload_is_ignored</code>	Run the node. The test sends <code>opened_on</code> during inventory-lot creation.	Lot creation succeeds, but returned <code>opened_on</code> is <code>None</code> .
API-018 <code>tests/test_api.py::test_inventory_adjustment_can_deplete_and_restore_lot</code>	Run the node. The test depletes a lot with an adjustment, restores availability with a positive adjustment, and reads adjustment history.	Lot reaches zero availability, depleted/inactive; positive correction restores availability and clears depletion; history includes two adjustments.
API-019 <code>tests/test_api.py::test_inventory_adjustment_cannot_exceed_max_lot_quantity</code>	Run the node. The test attempts an overdraw and a fractional positive adjustment.	Both requests return <code>400</code> with <code>invalid_quantity</code> .

<code>tests/test_api.py::test_inventory_adjustment_rejects_overdraw_and_fractional_count</code>	fractional count adjustment.	
API-020 <code>tests/test_api.py::test_inventory_adjustment_is_blocked_while_reserved</code>	Run the node. The test reserves inventory through a batch, then attempts a lot adjustment.	Adjustment returns <code>409</code> with <code>inventory_reserved</code> .
API-021 <code>tests/test_api.py::test_insufficient_inventory_rolls_back_batch</code>	Run the node. The test attempts to create a batch requiring more bullet inventory than available.	Batch creation fails with <code>400</code> or <code>409</code> ; batch list remains empty.
API-022 <code>tests/test_api.py::test_cancel_requires_explicit_return_and_loss</code>	Run the node. The test tries to cancel a reserved batch, accounts for every lot as returned/lost, then cancels again.	Direct cancellation returns <code>409</code> ; return/loss records succeed; cancellation succeeds after all reservations are accounted for.
API-023 <code>tests/test_api.py::test_production_loss_replaces_reserved_material_from_compatible_lot</code>	Run the node. The test records powder loss against an outstanding reservation and chooses an inactive compatible replacement lot, then completes production.	Source reservation is reduced; source lot records consumed loss; replacement reservation is created; production completion consumes both recipe reservations and promotes the replacement lot.
API-024 <code>tests/test_api.py::test_production_loss_can_replace_from_source_lot_available_inventory</code>	Run the node. The test records powder loss without a replacement lot when the source lot has extra available inventory.	The source lot supplies the replacement reservation, keeps total recipe reservation intact, records the loss as consumed, and reduces available inventory by the loss.
API-025 <code>tests/test_api.py::test_full_production_loss_depletes_source_and_promotes_replacement</code>	Run the node. The test records a full powder-reservation loss and chooses an inactive compatible replacement lot, then completes production.	Source reservation is marked <code>REPLACED</code> ; source lot is depleted/inactive immediately; selected replacement is promoted/opened and later consumed at production completion.
API-026 <code>tests/test_api.py::test_production_loss_validates_replacement_lot_and_units</code>	Run the node. The test attempts incompatible replacement, fractional count loss, and loss with no available replacement inventory.	Invalid replacement returns <code>400</code> ; fractional count loss returns <code>400</code> ; missing replacement inventory returns <code>409</code> .

Domain Helper Test Matrix

Purpose: these tests verify unit conversion, count validation, slug generation, slug-list capacity, and transition validation.

ID / pytest node	Test step	Verification
DOM-001 <code>tests/test_domain.py::test_powder_unit_conversion</code>	Run the node. The test normalizes powder pounds, ounces, and grams.	<code>1</code> pound becomes <code>7000.000000</code> grains; <code>1</code> ounce becomes <code>437.500000</code> grains; <code>10</code> grams becomes <code>154.323584</code> grains.
DOM-002 <code>tests/test_domain.py::test_count_conversion_requires_whole_count</code>	Run the node. The test normalizes a whole count and attempts a fractional count.	Whole count is accepted; fractional count raises <code>DomainError</code> with <code>invalid_quantity</code> .
DOM-003 <code>tests/test_domain.py::test_slug_collision_regenerates</code>	Run the node. The test forces a slug collision sequence.	Colliding <code>craft-anvil</code> is skipped; generated slug becomes <code>forge-ridge</code> .
DOM-004 <code>tests/test_domain.py::test_slug_word_lists_have_64_by_64_capacity</code>	Run the node. The test inspects slug word lists.	Verb and noun lists each contain 64 unique values; total capacity is 4096 combinations.

DOM-005 tests/test_domain.py::test_invalid_transition_is_rejected	Run the node. The test attempts an invalid lifecycle transition.	Transition raises <code>DomainError</code> with <code>invalid_transition</code> .
--	--	---

MCP Server Bridge Test Matrix

Purpose: these tests verify the Model Context Protocol bridge exposes the correct tools, handles authentication safely, validates tool arguments, and maps API errors correctly.

ID / pytest node	Test step	Verification
MCP-001 tests/test_mcp_server.py::test_initialize_declares_tools_capability	Run the node. The test sends a JSON-RPC <code>initialize</code> request.	Response returns protocol version, tool capability, and server name <code>reload-ledger-api</code> .
MCP-002 tests/test_mcp_server.py::test_tools_list_exposes_api_bridge_tools	Run the node. The test calls <code>tools/list</code> .	Tool names include <code>login</code> , <code>api_routes</code> , <code>api_get</code> , <code>api_post</code> , <code>api_patch</code> , <code>api_put</code> , and <code>api_delete</code> .
MCP-003 tests/test_mcp_server.py::test_login_stores_token_without_returning_it	Run the node. The test performs a fake login through the MCP bridge.	Login calls <code>/api/auth/login</code> ; token is stored internally; token is not exposed in tool text; login request has no authorization header.
MCP-004 tests/test_mcp_server.py::test_api_get_calls_relative_path_with_bearer_token_and_query	Run the node. The test calls <code>api_get</code> with path query and explicit query arguments.	URL is normalized to the API base; bearer token is included; both query sources are preserved.
MCP-005 tests/test_mcp_server.py::test_build_server_from_env_uses_auth_token	Run the node. The test sets MCP environment variables and builds the server.	Base URL, token, and timeout are loaded and normalized correctly.
MCP-006 tests/test_mcp_server.py::test_whoami_rejects_unexpected_arguments	Run the node. The test passes an unexpected argument to <code>whoami</code> .	JSON-RPC error <code>-32602</code> is returned and names the unexpected argument.
MCP-007 tests/test_mcp_server.py::test_api_path_must_not_be_absolute_url	Run the node. The test passes an absolute URL as an API path.	JSON-RPC error <code>-32602</code> is returned and says the path must be relative.
MCP-008 tests/test_mcp_server.py::test_api_errors_are_tool_execution_errors	Run the node. The test simulates an API <code>409</code> error through a tool call.	Tool result has <code>isError: True</code> ; status code and API error code are preserved.
MCP-009 tests/test_mcp_server.py::test_stdio_writes_newline_delimited_json_rpc_messages	Run the node. The test feeds initialize, notification, and tools/list JSON-RPC messages into stdio runner.	Two newline-delimited responses are emitted for request messages; notification emits no response; server initializes.

Renderer, Template, and Static-Script Test Matrix

Purpose: these tests verify rendered HTML, route filtering, static JavaScript behavior, authenticated download behavior, and UI state controls.

ID / pytest node	Test step	Verification
------------------	-----------	--------------

REN-001 tests/test_templates.py::test_dashboard_renders_item_count_in stead_of_dict_method	Run the node. The test renders dashboard metrics.	HIML shows numeric item count; does not show Python built-in method items; versioned CSS link is present.
REN-002 tests/test_templates.py::test_authenticated_topbar_includes_help_menu	Run the node. The test renders dashboard with an authenticated session.	Help menu, help video links, core nav links, and LLM context download link are present.
REN-003 tests/test_templates.py::test_item_form_marks_category_specific_fields	Run the node. The test renders the items page.	Category-specific data attributes render; advanced item attributes section is ordered correctly; versioned items.js is included.
REN-004 tests/test_templates.py::test_item_form_script_uses_category_specific_placeholders	Run the node. The test reads rendering_app/static/items.js.	Script includes category-specific placeholders and realistic item examples.
REN-005 tests/test_templates.py::test_item_table_uses_only_universal_columns	Run the node. The test renders an item table.	Universal columns render; category-specific values are excluded from the universal table body.
REN-006 tests/test_templates.py::test_existing_records_render_before_creation_forms	Run the node. The test renders items, inventory, and recipes pages with existing records.	Existing records appear before creation forms on all three pages.
REN-007 tests/test_templates.py::test_containers_render_before_creation_form_with_batch_quantities	Run the node. The test renders containers with assigned and assignable batches.	Existing container appears before form; capacity and assignments render; full batches are hidden; mixed-batch acknowledgement controls and containers.js render.
REN-008 tests/test_templates.py::test_container_script_toggles_mixed_batch_acknowledgement_and_auto_submits_state	Run the node. The test reads containers.js.	Script auto-submits lifecycle state changes and toggles mixed-batch acknowledgement controls.
REN-009 tests/test_templates.py::test_inventory_form_uses_expanded_item_picker_for_lot_creation	Run the node. The test renders inventory lot creation UI.	Item type picker renders; completed-cartridge option is absent; active-lot marker, item details, replacement field, and inventory.js render.
REN-010 tests/test_templates.py::test_inventory_script_uses_item_type_specific_placeholders	Run the node. The test reads inventory.js.	Script includes lot placeholders for bullet, powder, primer, case, and other item categories.
REN-011 tests/test_templates.py::test_recipe_component_form_derives_role_from_item	Run the node. The test renders recipe detail component UI.	Role input and alternative group are absent; item/category explanation renders; missing-source controls render; component form opens via query parameter.
REN-012 tests/test_templates.py::test_recipe_lifecycle_select_reflects_current_state	Run the node. The test renders recipe lifecycle select.	Current state UNDER_TEST is selected and other states are not incorrectly selected.
REN-013 tests/test_templates.py::test_batch_lifecycle_select_includes_and_selects_under_production	Run the node. The test renders batch detail in under-production, produced, and depleted states.	Manual transition options are valid; storage-derived states are not manual choices; performance form appears only after production; depleted batch omits decommission

		option.
REN-014 <code>tests/test_templates.py::test_batch_detail_script_auto_submits_lifecycle_changes</code>	Run the node. The test reads <code>batch-detail.js</code> .	Script references batch state form/select and submits the form when state changes.
REN-015 <code>tests/test_templates.py::test_batches_table_rows_are_clickable</code>	Run the node. The test renders batch table rows.	Clickable row attributes, recipe link, depleted toggle link, and clickable-row script render.
REN-016 <code>tests/test_templates.py::test_batches_template_toggle_hides_depleted_batches_by_default</code>	Run the node. The test renders depleted batch toggle state.	Link text says <code>Hide depleted</code> for <code>?depleted=true</code> .
REN-017 <code>tests/test_templates.py::test_clickable_row_script_suppresses_row_hover_on_links</code>	Run the node. The test reads clickable row JS and CSS.	Script sets <code>link-hover</code> ; CSS suppresses clickable-row hover background while hovering links.
REN-018 <code>tests/test_templates.py::test_qr_page_has_back_link_and_embedded_png</code>	Run the node. The test renders QR label page.	Back link targets batch page; QR image source targets <code>/download/qr/batch/<id></code> .
REN-019 <code>tests/test_templates.py::test_llm_context_download_is_protected_text_file</code>	Run the node. The test requests LLM context unauthenticated and authenticated.	Unauthenticated request redirects to login; authenticated response is <code>text/plain</code> , has expected filename, and includes expected context phrases.
REN-020 <code>tests/test_templates.py::test_settings_page_shows_current_session_api_token</code>	Run the node. The test requests settings unauthenticated and authenticated.	Unauthenticated request redirects to login; authenticated page displays API token, env var assignment, and expiration.
REN-021 <code>tests/test_templates.py::test_incomplete_component_submission_redirects_to_open_form</code>	Run the node. The test mocks component API response with missing-component warnings.	Redirect ends with <code>/recipes/recipe-id?component_form=open#add-component</code> .
REN-022 <code>tests/test_templates.py::test_complete_component_submission_redirects_to_collapsed_form</code>	Run the node. The test mocks component API response with only missing-source warning.	Redirect ends with <code>/recipes/recipe-id#components</code> and omits <code>component_form=open</code> .
REN-023 <code>tests/test_templates.py::test_recipe_creation_form_uses_examples_without_submitted_default_title</code>	Run the node. The test renders recipe creation form.	Title input uses example placeholder; submitted default title is absent; legacy suggested slug text is absent.
REN-024 <code>tests/test_templates.py::test_recipe_identifier_is_explicitly_labeled</code>	Run the node. The test renders recipe list.	<code>Recipe ID:</code> label, UUID code value, and retired toggle link render.
REN-025 <code>tests/test_templates.py::test_recipes_template_toggle_hides_retired_recipes_by_default</code>	Run the node. The test renders retired recipe toggle state.	Link text says <code>Hide retired</code> for <code>?retired=true</code> .
REN-026 <code>tests/test_templates.py::test_recipes_route_hides_retired_recipes_by_default</code>	Run the node. The test mocks recipe list API and requests default/toggled route states.	Default route shows active and hides retired; toggled route shows both; toggle link changes to <code>Hide retired</code> .

ords_until_toggled		
REN-027 tests/test_templates.py::test_batches_route_hides_depleted_re ords_until_toggled	Run the node. The test mocks batch list API and requests default/toggled route states.	Default route shows active and hides depleted; toggled route shows both; toggle link changes to Hide depleted.
REN-028 tests/test_templates.py::test_batch_form_derives_required_qua ntities_from_recipe	Run the node. The test renders batch creation form and calls recipe_allocations.	Form uses lot selector and component quantity data attributes; legacy manual quantity input is absent; derived allocation equals recipe quantity times iterations.

Selenium End-to-End Browser Test Matrix

Purpose: this test verifies the major user workflows through the rendered browser interface, including duplicate core-component prevention, the user-visible automatic replacement-lot flow, Garmin FIT performance import, and successor-lot promotion behavior.

Node ID: tests/e2e/test_357_magnum_workflow.py::test_357_magnum_browser_workflow

Headless command:

```
docker compose --profile selenium up --build -d
docker compose run --rm \
  -e APP_BASE_URL=http://web:8080 \
  -e SELENIO_REMOTE_URL=http://selenium:4444/wd/hub \
  storage pytest --run-selenium tests/e2e/test_357_magnum_workflow.py::test_357_magnum_browser_workflow -q
```

Visible command:

```
docker compose --profile selenium up --build -d
docker compose run --rm \
  -e APP_BASE_URL=http://web:8080 \
  -e SELENIO_REMOTE_URL=http://selenium:4444/wd/hub \
  -e SELENIO_HEADLESS=false \
  -e SELENIO_SLOW_MS=350 \
  storage pytest --run-selenium tests/e2e/test_357_magnum_workflow.py::test_357_magnum_browser_workflow -q
```

In visible mode, open http://localhost:7900.

Step	Browser/operator-observable test step	Verification
1	Register a unique test account.	Account creation success message appears.
2	Attempt login with wrong password.	Error message says email or password is incorrect.
3	Login, logout, and login again.	Authenticated pages load, then login page returns after logout, then authenticated pages load again.
4	Create ten item definitions covering bullets, powders, primers, cases, and support items.	Items page shows created item names; dashboard item metric is 10.
5	Create two inventory lots per item, including active and reserve lots.	Inventory page shows created lots; dashboard current-lot metric is 20.
6	Create two approved .357 Magnum recipes.	Recipe detail pages show APPROVED.
7	Open the add-component workflow after all core roles are present.	Browser offers only non-core/support items; duplicate Bullet options are not selectable.
8	Create and open a public recipe link.	Public page opens and shows the public recipe content.

9	Create a dedicated successor-lot promotion recipe.	Recipe detail page shows the recipe approved.
10	Use the automatic replacement-lot row to consume all of <code>HDY-125-ACT</code> and continue into <code>HDY-125-RES</code> .	Batch creation succeeds and page shows <code>UNDER PRODUCTION</code> .
11	Before production, inspect Inventory.	<code>HDY-125-ACT</code> is visible as active with zero available count reserved; <code>HDY-125-RES</code> is visible as inactive/unopened with consumed reservation.
12	Transition the promotion batch to <code>PRODUCED</code> .	Browser shows batch state changed to <code>PRODUCED</code> .
13	Inspect Inventory after production.	<code>HDY-125-ACT</code> is visible as depleted; <code>HDY-125-RES</code> is visible as active and has an opened date.
14	Inspect Audit after promotion.	Audit page contains <code>PROMOTED</code> .
15	Create three production batches from the primary recipes and transition them to <code>PRODUCED</code> .	Each batch detail page shows <code>PRODUCED</code> .
16	Save a performance record for the first batch and import Garmin FIT data.	Browser shows the record saved, Garmin import succeeds, imported chronograph fields become read-only, and editable field values are preserved.
17	Create two storage containers.	Containers page shows the created identifiers.
18	Assign batches to containers and attempt one overfill.	Valid assignments succeed; overfill shows assignment-limit error.
19	Assign mixed-batch container contents with acknowledgement.	Mixed-batch assignment succeeds after acknowledgement.
20	Inspect batch storage states.	Batches show expected <code>IN STORAGE</code> and <code>PARTIALLY IN STORAGE</code> quantities.
21	Transition containers through <code>PARTIALLY USED</code> , <code>USED</code> , and <code>EMPTY</code> .	Containers show lifecycle changes and eventually empty state.
22	Inspect final batch states.	Fully containerized batches show <code>DEPLETED</code> ; partially containerized batch shows <code>PARTIALLY DEPLETED</code> .
23	Inspect Audit for state-change activity.	Audit page contains <code>STATE_CHANGED</code> .

Suite-Level Acceptance Criteria

Acceptance item	Verification evidence
Test inventory is current.	<code>docker compose run --rm storage pytest --collect-only -vv</code> reports <code>65 tests collected</code> .
Non-Selenium tests pass.	<code>docker compose run --rm storage pytest -q</code> exits <code>0</code> and reports the Selenium test skipped by default.
Selenium tests pass.	The Selenium command exits <code>0</code> with the E2E test passing.
Browser workflow is observable when needed.	Visible-mode run can be watched at <code>http://localhost:7900</code> .
Failures are triaged.	Record node ID, command, output, browser page text if supplied, relevant logs, and git revision/local diff.
Documentation is reproducible	Regenerate HTML with <code>hander docs/Test Execution Guide.md --standalone --toc --</code>

Documentation is reproducible.

```
regenerate HTML with pandoc docs/Test_Execution_Guide.md --standalone --toc --css=/home/swiseman/repositories/reloading_app/docs/Test_Execution_Guide.css -o /tmp/Test_Execution_Guide.html, then PDF with wkhtmltopdf --enable-local-file-access --orientation Landscape --page-size Letter --margin-top 8mm --margin-right 8mm --margin-bottom 8mm --margin-left 8mm /tmp/Test_Execution_Guide.html docs/Test_Execution_Guide.pdf.
```